Programmazione procedurale

Appunti

Marco Cerbella

Università degli Studi di Perugia – A.A. 2026/2027
Aggiornato al 13 ottobre 2026

Indice

1 Algoritmi e diagrammi di flusso (Lezione 1 – 22 settembre 2026) 2

2 Esercizi sui diagrammi di flusso (Lezione 2 – 23 settembre 2026) 4

3 Il linguaggio C (Lezione 3 – 28 settembre 2026) 7

4 Struttura di un programma C e compilazione (Lezione 4 – 29 settembre 2026) 9

5 Errori di compilazione e di esecuzione (Lezione 5 – 30 settembre 2026) 12

6 Commenti, identificatori e parole chiave (Lezione 6 – 5 ottobre 2026) 13

7 Scope e binding (Lezione 7 – 6 ottobre 2026) 15

8 Tipi di dato (Lezione 8 – 7 ottobre 2026) 18

9 Costanti letterali (Lezione 9 – 12 ottobre 2026) 25

10 Conversioni di tipo (Lezione 10 – 13 ottobre 2026) 27

1 Algoritmi e diagrammi di flusso

Lezione 1 – 22 settembre 2026

1.1 Algoritmi e diagrammi di flusso

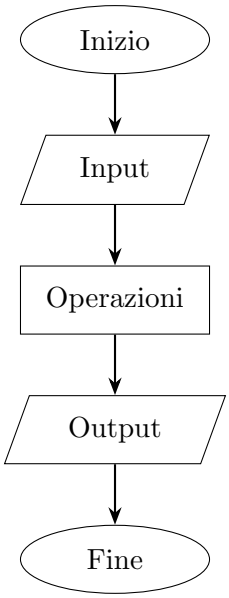
Definizione. Un *algoritmo* è una procedura che risolve un problema attraverso una sequenza finita di passi elementari e non ambigui. Spesso comporta la ripetizione di alcune operazioni.

Esempio. Il primo algoritmo celebre è quello di Euclide (*Elementi*, 300 a.C.) per il massimo comun divisore.

Definizione. Un *diagramma di flusso* (*flowchart*) è la rappresentazione grafica di un algoritmo. Le operazioni si leggono dall'alto verso il basso, seguendo le frecce. I simboli sono standardizzati (ANSI/ISO).

I simboli principali sono:

- **ellisse**: inizio e fine;
- **parallelogramma**: input e output (I/O);
- **rettangolo**: operazione (*processo*);
- **rombo**: decisione (condizione vero/falso).



Lo stesso schema in linguaggio *C*:

```c
// Importare le librerie necessarie
#include <stdio.h>

// Funzione principale
int main() {
    // Lettura dell'input e operazioni

    // Stampa del risultato
```

```
    return 0;
}
```

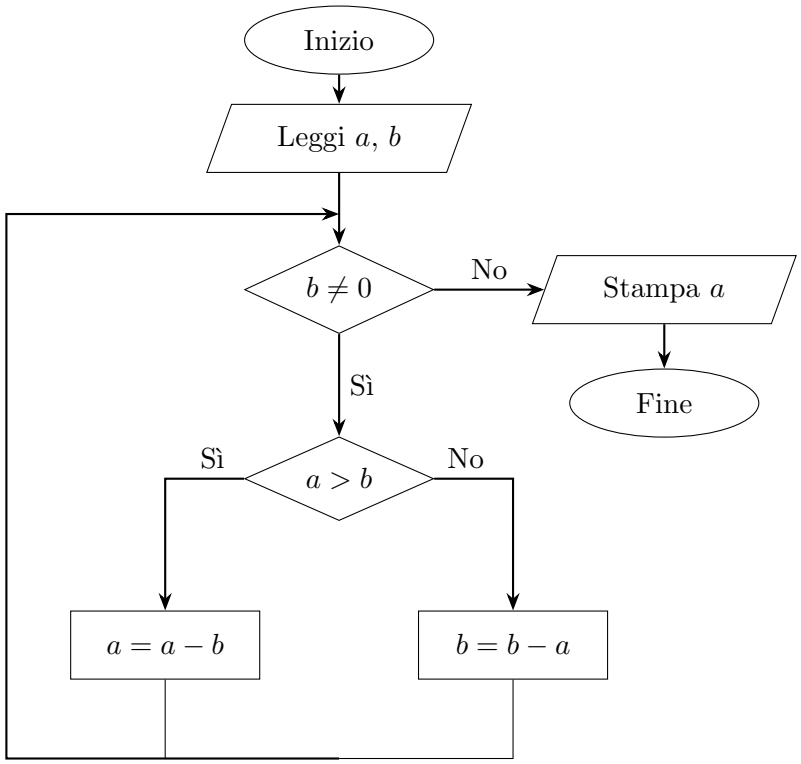
1.2 Programmare

Definizione. *Programmare* significa tradurre un algoritmo in una notazione, un linguaggio di programmazione, che un elaboratore può eseguire. Un *programma* è un insieme di istruzioni.

I passi sono: partire dal problema, *pensare all'algoritmo*, scegliere il linguaggio giusto e solo dopo programmare.

1.3 Esempio: massimo comun divisore

Algoritmo di Euclide (versione con sottrazioni): finché $b \neq 0$, si sostituisce il maggiore tra a e b con la differenza tra i due. Alla fine a contiene il MCD.



E il programma C corrispondente:

```c
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter first positive integer: \n");
    scanf("%d", &a);
    printf("Enter second positive integer: \n");
    scanf("%d", &b);

```

```
    while (b != 0) {
        if (a > b)
            a = a - b;
        else
            b = b - a;
    }
    printf("GCD = %d\n", a);

    return 0;
}
```

Osservazione. Il ciclo `while` corrisponde al rombo $b \neq 0$ con la freccia che torna indietro; l'`if`/`else` corrisponde al secondo rombo.

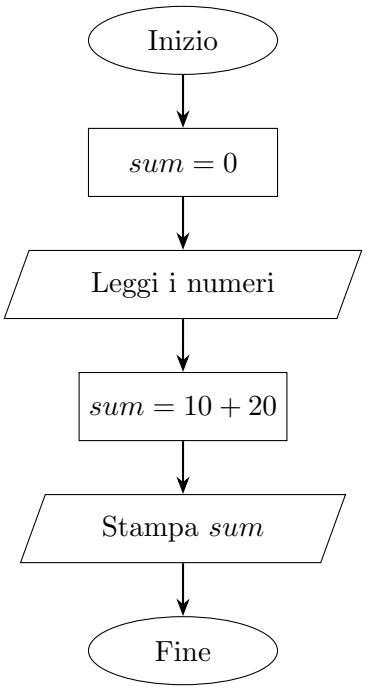
2 Esercizi sui diagrammi di flusso

Lezione 2 – 23 settembre 2026

2.1 Somma di 10 e 20

Si usa una variabile `sum` inizializzata a zero; si leggono i due numeri, si sommano salvando il risultato in `sum` e infine si stampa.

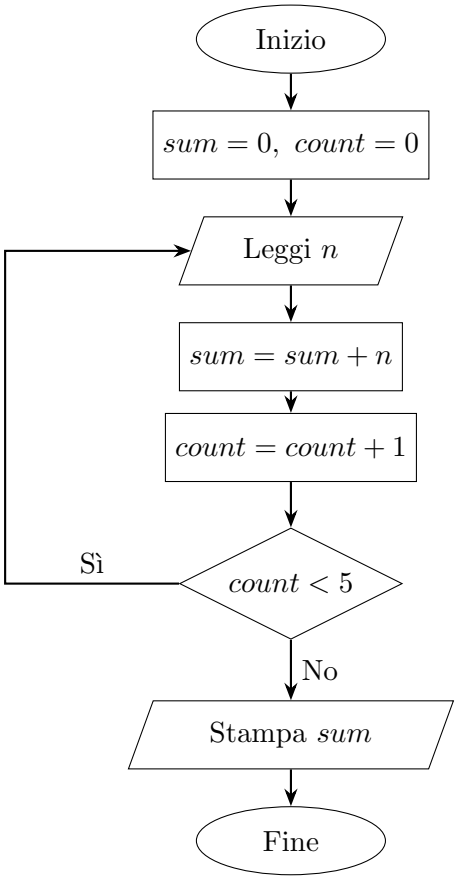
1. Inizializza $sum = 0$ (processo)
2. Leggi i numeri (I/O)
3. $sum = 10 + 20$ (processo)
4. Stampa sum (I/O)



2.2 Somma di 5 numeri

Servono due variabili: `sum` (il risultato) e `count` (quanti numeri sono stati letti). Si usa un *ciclo*: i passi si ripetono finché la condizione è vera.

1. $sum = 0$, $count = 0$ (processo)
2. Leggi n (I/O)
3. $sum = sum + n$ e $count = count + 1$ (processo)
4. $count < 5$? (decisione)
5. Se sì, torna al passo 2; altrimenti stampa sum (I/O)



Il programma C corrispondente:

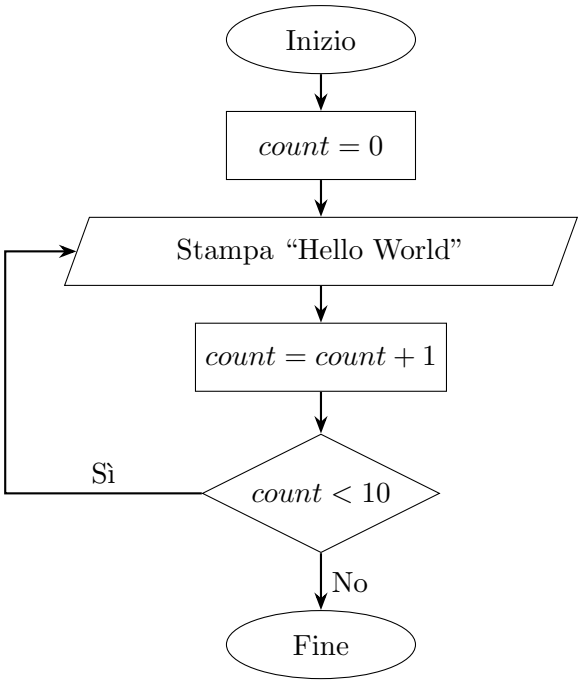
```c
#include <stdio.h>

int main() {
    int sum = 0, count = 0, n;
    while (count < 5) {
        scanf("%d", &n);
        sum = sum + n;
        count = count + 1;
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

2.3 Stampare “Hello World” 10 volte

Anche qui si usa un ciclo con un contatore.

1. $count = 0$ (processo)
2. Stampa “Hello World” (I/O)
3. $count = count + 1$ (processo)
4. $count < 10$? Se sì torna al passo 2, altrimenti termina.




```c
#include <stdio.h>

int main() {
    int count = 0;
    do {
        printf("Hello World\n");
        count = count + 1;
    } while (count < 10);
    return 0;
}
```

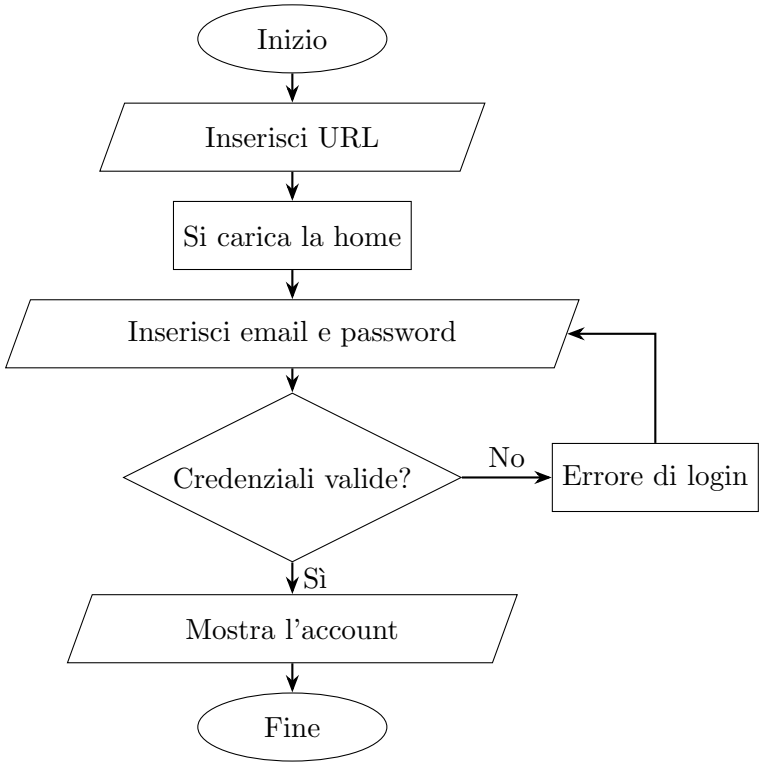
Osservazione. Il diagramma esegue la stampa *prima* di controllare la condizione, quindi il costrutto C fedele è `do ... while`. Con un `while` la condizione verrebbe controllata prima del corpo.

2.4 Login a un account (da risolvere liberamente)

Esercizio aperto: non esiste una sola soluzione. Una possibile sequenza:

1. Inserisci l’indirizzo del sito nel browser (I/O)
2. Si carica la home page (processo)
3. Inserisci email e password (I/O)
4. Credenziali valide? (decisione)

5. Se no: errore di login (processo) e torna al passo 3; se sì: mostra l'account (I/O) e termina.



3 Il linguaggio C

Lezione 3 – 28 settembre 2026

3.1 Linguaggi di programmazione

Definizione. Un *linguaggio di programmazione* è un linguaggio formale che specifica un insieme di istruzioni per un elaboratore. Si usa per realizzare programmi che implementano algoritmi. I linguaggi *ad alto livello* sono vicini al linguaggio umano e lontani dal linguaggio macchina.

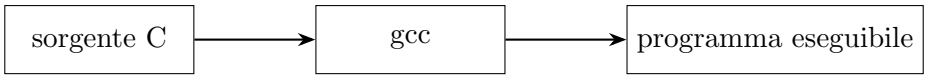
I *paradigmi di programmazione* classificano i linguaggi in base alle loro caratteristiche. Il C è *imperativo*: il programmatore indica alla macchina *come* modificare il proprio stato, ad esempio `a = a + 1;`.

3.2 Dal linguaggio macchina al C

- Il *linguaggio macchina* (sequenze di bit) è l'unico eseguito direttamente dal processore.
- L'*assembly* usa abbreviazioni mnemoniche per le operazioni, ma è legato alla macchina.
- I linguaggi *ad alto livello* sono indipendenti dalla macchina.

Definizione. Un *compilatore* è un programma di sistema che traduce il programma sorgente scritto in un linguaggio ad alto livello in un programma *target* equivalente (assembly/linguaggio macchina). Il programma target viene eseguito in un secondo momento.

In questo corso il compilatore è **gcc**.



3.3 Le fasi della compilazione con gcc

Quando si lancia `gcc`, il passaggio da sorgente a eseguibile non è un'unica operazione ma una catena di quattro fasi, ciascuna delle quali può essere fermata con una opzione.

Fase	**Comando**	**Risultato**	**Cosa fa**
Preprocessing	`gcc -E main.c -o main.i`	`main.i`	gestisce le righe che iniziano con `#`: sostituisce i `#define`, inserisce il contenuto degli `#include`, toglie i commenti. Il risultato è ancora codice C.
Compilazione	`gcc -S main.c`	`main.s`	traduce il C in *assembly*. Qui emergono gli errori di sintassi e di tipo.
Assemblaggio	`gcc -c main.c`	`main.o`	traduce l'assembly in codice macchina (*file oggetto*). Non è ancora eseguibile.
Linking	`gcc main.o -o prog`	`prog`	unisce i file oggetto e le librerie e produce l'eseguibile.

Osservazione. Un file oggetto contiene codice macchina ma le chiamate a funzioni definite in altri file o in libreria (ad esempio `printf`) non hanno ancora un indirizzo. Il *linker* li risolve. Un errore di linking tipico è `undefined reference to 'fun'`: la funzione è stata dichiarata ma non è stata definita in nessuno dei file collegati.

I file oggetto servono per compilare ogni file sorgente separatamente e unirli alla fine. Con due file `main.c` e `altro.c`:

```
gcc -c main.c
gcc -c altro.c
gcc -o prog main.o altro.o
```

oppure in un solo comando `gcc -o prog main.c altro.c`.

3.4 Caratteristiche del C

Il C è un linguaggio ad alto livello, *procedurale* e di uso generale, progettato (Dennis Ritchie, anni '70) per scrivere sistemi operativi con la massima indipendenza dall'hardware.

Pregi:

- veloce (compilato, vicino alla macchina) e portabile;
- linguaggio piccolo e maturo, con molti strumenti;
- accesso diretto alla memoria e alle funzionalità di basso livello.

Difficoltà:

- si può facilmente fare danni con accessi diretti alla memoria e puntatori;
- la memoria va gestita a mano;
- il codice è più verboso rispetto a linguaggi di scripting come Python.

Osservazione. Il linguaggio C in sé non ha istruzioni di input/output né di gestione della memoria dinamica: queste funzioni sono fornite dalla *libreria standard* (ad esempio `stdio.h`).

3.5 Standard

Il C è stato standardizzato più volte: C89/C90 (ANSI/ISO), C99, C11, C18. Con gcc si può scegliere lo standard:

```
gcc file.c -std=c11
```

3.6 Compilati e interpretati

Un linguaggio *compilato* (come C) viene analizzato a fondo dal traduttore prima dell'esecuzione, e quasi tutti i controlli semantici sono fatti a tempo di compilazione. In un linguaggio *interpretato* (come Python) l'interprete resta presente durante l'esecuzione e la maggior parte dei controlli è fatta a tempo di esecuzione.

Osservazione. Un interprete non produce necessariamente codice macchina: spesso esegue direttamente le istruzioni (Python, ad esempio, compila prima in *bytecode* e poi lo interpreta). Essere compilato o interpretato è inoltre una proprietà dell'*implementazione* più che del linguaggio. Un vantaggio del compilato è che molti errori (sintassi, tipi) sono segnalati prima che il programma giri; con l'interprete molti emergono solo quando la riga viene eseguita.

4 Struttura di un programma C e compilazione

Lezione 4 – 29 settembre 2026

4.1 Struttura di un programma

Gli elementi costitutivi di un programma C sono le *funzioni*, che possono chiamarsi a vicenda.

- Ogni funzione ha uno scopo preciso; le funzioni *non* si possono annidare.
- Una funzione contiene istruzioni eseguite in sequenza; le istruzioni si raggruppano in *blocchi*.
- Si possono usare funzioni della libreria standard (`printf`) o scriverne di proprie.
- Ogni programma deve definire almeno la funzione `main()`: è la prima funzione eseguita ed è il livello principale di controllo.

4.2 Hello World e compilazione con gcc

```c
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```

Salvato il codice in `test.c`, da terminale si compila (creando l'eseguibile `test`) e poi si esegue:

```
gcc -o test test.c
./test
```

Osservazione. Per il corso e per l'esame si compila *solo con gcc* da terminale (anche se si usa un editor che ha un terminale integrato). Senza `-o` il file eseguibile si chiama `a.out`.

4.3 Input e output: printf e scanf

Esempio (Somma di due interi)**.**

```c
#include <stdio.h>
int main() {
    int firstNumber, secondNumber, sumOfTwoNumbers;

    printf("Enter two integers: ");

    // gli interi inseriti vengono memorizzati con scanf()
    scanf("%d %d", &firstNumber, &secondNumber);

    // la somma viene salvata in sumOfTwoNumbers
    sumOfTwoNumbers = firstNumber + secondNumber;

    // stampa della somma
    printf("%d + %d = %d", firstNumber, secondNumber, sumOfTwoNumbers);

    return 0;
}
```

```
Enter two integers: 12
11
12 + 11 = 23
```

Esempio (Quoziente e resto)**.**

```c
#include <stdio.h>
int main() {
    int dividend, divisor, quotient, remainder;

    printf("Enter dividend: ");
    scanf("%d", &dividend);

    printf("Enter divisor: ");
    scanf("%d", &divisor);

    quotient = dividend / divisor;    // quoziente
    remainder = dividend % divisor;   // resto

```

```c
    printf("Quotient = %d\n", quotient);
    printf("Remainder = %d", remainder);

    return 0;
}
```

```
Enter dividend: 25
Enter divisor: 4
Quotient = 6
Remainder = 1
```

Osservazione. `%d` è il segnaposto per un intero; in `scanf` le variabili vanno precedute da `&`.

4.4 Programmazione procedurale e modulare

Definizione. La *programmazione procedurale* è un paradigma, derivato dalla programmazione strutturata, basato sul concetto di chiamata di procedura. Le procedure (o routine, sottoprogrammi, funzioni) contengono una serie di passi di calcolo da eseguire. Il C è imperativo e procedurale: le istruzioni sono raggruppate in funzioni.

Esempio. `mainfile.c`

```c
#include <stdio.h>
void myPrintHello();

int main() {
    myPrintHello();
    return(0);
}

void myPrintHello(void) {
    printf("Hello!\n");
    return;
}
```

Definizione. La *programmazione modulare* è una tecnica di progettazione che separa le funzionalità di un programma in *moduli* indipendenti e intercambiabili, ciascuno contenente tutto il necessario per realizzare un solo aspetto della funzionalità.

Ad esempio il codice si può dividere in file diversi.

`mainfile.c`:

```c
int main() {
    myPrintHello();
    return(0);
```

```
}
```

`hello.c`:

```c
#include <stdio.h>

void myPrintHello(void) {
    printf("Hello!\n");
    return;
}
```

e si compilano insieme:

```
gcc -o executable mainfile.c hello.c
```

5 Errori di compilazione e di esecuzione

Lezione 5 – 30 settembre 2026

5.1 Compile-time e run-time

La vita di un programmatore ha due momenti: il *compile-time*, quando si esegue `gcc`, e il *run-time*, quando si esegue il programma.

Definizione. Un *errore di compilazione* (*compile-time*) è un errore rilevato dal compilatore. Cause comuni: errori di sintassi, come un punto e virgola mancante o l'uso di una parola riservata (ad esempio `while`) come nome.

Definizione. Un *errore di esecuzione* (*run-time*) è un errore del programma che si verifica mentre il programma è in esecuzione.

5.2 Esempio di errore di compilazione

Nel programma del MCD manca il `;` dopo la seconda `scanf`:

```c
printf("Enter first positive integer: \n");
scanf("%d", &b)        // manca il ';'
```

Il compilatore segnala:

```
$ gcc -o euclid euclid.c
euclid.c:9:20: error: expected ';' after expression
   scanf("%d", &b)
                  ^
                  ;
1 error generated.
```

Il messaggio indica file, riga e colonna dell'errore: bisogna imparare a leggere l'output del compilatore per eliminare tutti gli errori di compilazione.

5.3 Esempio di errore di esecuzione

Il programma compila senza problemi, ma dereferenzia un puntatore nullo:

```c
#include <stdio.h>
int main() {
    int *a = NULL;

    printf("%d", *a);    // accesso a memoria non valida

    int i = 0;
    while (i < 4) {
        printf("Hello World");
        i++;
    }
    return 0;
}
```

```
$ gcc -o runtime_error runtime_error.c
$ ./runtime_error
Segmentation fault: 11
```

Osservazione. Eliminare gli errori di compilazione non basta: bisogna testare il programma con molti input diversi per eliminare (il più possibile) gli errori di esecuzione.

"Testing shows the presence, not the absence of bugs" – E. W. Dijkstra

6 Commenti, identificatori e parole chiave

Lezione 6 – 5 ottobre 2026

6.1 Commenti

In C ci sono due tipi di commenti:

- *a blocco*: iniziano con `/*` e finiscono con `*/`, possono occupare più righe;
- *di riga*: iniziano con `//` e finiscono a fine riga.

```c
double pi = 3.1415926536;   // Pi è 3,14
```

All'interno di una stringa o di una costante carattere i simboli `/*` e `//` non iniziano un commento:

```c
printf("Comments in C begin with /* or //.\n");
```

Osservazione. I commenti a blocco *non si possono annidare*: il primo `*/` chiude il commento aperto. Si possono invece usare per "spegnere" del codice che contiene commenti di riga.

Esempio. Qui il commento aperto alla riga 3 termina al primo `*/` (riga 6): il secondo `*/` è un errore di sintassi.

```
1 #include <stdio.h>
2 int main() {
3     int x = 4; /*
4     int y = 10; /*
5
6     printf("x = %d\n", x); */
7     printf("y = %d\n", y); */
8 }
```

6.1.1 Scrivere buoni commenti

Il codice si scrive per essere eseguito da un computer ma letto da persone: da noi stessi mentre lo scriviamo e dopo settimane o mesi, dai colleghi che devono integrarlo e da chi lo manterrà anni dopo.

- Evitare commenti ridondanti, come `i = i + 1; // incrementa i`.
- Non lasciare codice "eliminato" sotto forma di commento (va bene solo temporaneamente durante il debug).
- Evitare commenti decorati (riquadri di asterischi): sono difficili da mantenere.
- Un commento che chiarisce spesso segnala codice troppo complesso: meglio semplificare il codice, che deve essere *autoesplicativo*.

6.2 Caratteri

Il C definisce due insiemi di caratteri: quello *sorgente* (usabile nel codice) e quello di *esecuzione* (interpretabile dal programma in esecuzione); nella maggior parte delle implementazioni coincidono. I caratteri *spazio bianco* sono: spazio, `\f` (form feed), `\n` (a capo), `\t` (tab orizzontale), `\v` (tab verticale).

6.3 Identificatori

Definizione. Un *identificatore* è il nome di una variabile, funzione, macro, struttura o altro oggetto definito in un programma C. Può contenere:

- lettere `a--z` e `A--Z` (il C è *case-sensitive*);
- il carattere underscore _;
- cifre `0--9`, ma il primo carattere *non* può essere una cifra.

Esempio. Identificatori validi: `x`, `dollar`, `While`, `error_handler`, `scale64`.

Non validi: `1st_rank` (inizia con una cifra), `switch` (parola riservata), `y/n` e `x-ray` (caratteri non ammessi).

Non c'è un limite alla lunghezza, ma i compilatori considerano significativi solo i primi 31 o 63 caratteri. Molti nomi sono già usati dalla libreria standard (ad esempio `printf`) e non si possono riutilizzare per funzioni proprie o variabili globali.

6.4 Parole riservate

Le seguenti 44 parole chiave sono riservate in C (erano 32 in C89), hanno un significato preciso per il compilatore e non si possono usare come identificatori:

```
auto, break, case, char, const, continue, default, do, double, else, enum,
extern, float, for, goto, if, inline, int, long, register, restrict, return,
short, signed, sizeof, static, struct, switch, typedef, union, unsigned,
void, volatile, while, _Alignas, _Alignof, _Atomic, _Bool, _Complex, _Generic,
_Imaginary, _Noreturn, _Static_assert, _Thread_local
```

6.4.1 Scegliere i nomi

Un nome descrive l'oggetto, il suo comportamento e il suo uso: una variabile chiamata male può confondere molto.

- evitare nomi inutilmente lunghi;
- usare nomi diversi in ambiti diversi, per evitare confusione.

Esempio. Confrontare `a = b * c;` con `weekly_pay = hours_worked * pay_rate;`.

7 Scope e binding

Lezione 7 – 6 ottobre 2026

7.1 Scope degli identificatori

Definizione. Lo *scope* (ambito) di un identificatore è la parte del programma in cui l'identificatore è significativo, cioè in cui può essere "visto". Il tipo di scope dipende da dove l'identificatore viene dichiarato.

- **File scope**: l'identificatore è dichiarato fuori da tutti i blocchi e dalle liste di parametri (cioè fuori dalle funzioni). Si può usare dalla dichiarazione fino alla fine del file.
- **Block scope**: l'identificatore è dichiarato dentro un blocco. Si può usare dalla dichiarazione fino alla fine del blocco più piccolo che la contiene (spesso, ma non sempre, il corpo di una funzione).

Esempio.

```c
int p;                  // file scope

int fun2(void) {        // fun2: file scope
    int r = 3;          // r, s: block scope
    int s = r;
    while (r != s) {
        r--;
    }
    return r;
}
```

Osservazione. Non si possono avere due variabili con lo stesso nome nello stesso scope:

```c
int main() {
    int a = 5;
    float a = 6.5;   // errore: a già dichiarata
}
```

7.2 Shadowing

Si può riusare un identificatore in una dichiarazione *annidata* nello scope esistente; la nuova dichiarazione deve avere block scope, in un blocco contenuto in quello esterno. In questo caso la dichiarazione interna *nasconde* (*shadowing*) quella esterna, che non è visibile nello scope interno.

Esempio.

```c
int main() {
    int x = 1, y = 2, z = 3;
    printf(" x = %d, y = %d, z = %d \n", x, y, z);
    {
        int x = 10;
        float y = 20;
        printf(" x = %d, y = %f, z = %d \n", x, y, z);
        {
            int z = 100;
            printf(" x = %d, y = %f, z = %d \n", x, y, z);
        }
    }
    return 0;
}
```

```
x = 1, y = 2, z = 3
x = 10, y = 20.000000, z = 3
x = 10, y = 20.000000, z = 100
```

Esempio. Qui `x++` modifica la `x` esterna (non è ridichiarata), mentre `y++` modifica la `y` interna; uscendo dal blocco interno la `y` torna a valere 20.

```c
int main() {
    {
        int x = 10, y = 20;
        {
            printf("x = %d, y = %d\n", x, y);
            {
                int y = 40;
                x++;
                y++;
                printf("x = %d, y = %d\n", x, y);
            }
            printf("x = %d, y = %d\n", x, y);
```

```
        }
    }
    return 0;
}
```

```
x = 10, y = 20
x = 11, y = 41
x = 11, y = 20
```

Esempio. Fuori dal blocco in cui è dichiarata, una variabile non è accessibile:

```
int main() {
    {
        int x = 10;
    }
    {
        printf("%d", x);    // errore: x non visibile qui
    }
    return 0;
}
```

```
error: 'x' undeclared (first use in this function)
```

7.3 Binding statico

Definizione. Un *binding* è l'associazione tra un nome e l'oggetto che esso denota. Il *binding time* è il momento in cui il binding viene creato (più in generale, il momento in cui viene presa una decisione di implementazione). La regione testuale del programma in cui un binding è attivo è il suo scope.

In C lo scope di un binding è determinato *staticamente*, cioè a tempo di compilazione (*static scoping*), come si vede dal testo del programma e non dall'ordine delle chiamate a run-time. Altri linguaggi usano lo scoping *dinamico*: ad esempio in Perl `my` definisce una variabile locale a scope statico, `local` una a scope dinamico.

7.4 Esercizio

Esercizio. Qual è l'output del seguente programma?

```
#include <stdio.h>

int f1(void);
int f2(int x, int a);

int a;

```

```c
int main() {
    int a, b, c;

    a = 7;
    b = f1();
    c = f2(a, b);
    printf("%d %d %d\n", a, b, c);
}

int f1(void) {
    a = 12;
    printf("%d ", a);
    return (a + 5);
}

int f2(int x, int a) {
    printf("%d ", a);
    return (x * a);
}
```

Soluzione.

- In `main` la `a` locale (che vale 7) nasconde la `a` globale.
- `f1` non dichiara una propria `a`: per binding statico modifica quella *globale*, la pone a 12, stampa `12` e restituisce 17, quindi $b = 17$.
- In `f2(7, 17)` il parametro `a` vale 17: stampa `17` e restituisce $7 \cdot 17 = 119$, quindi $c = 119$.
- Infine `main` stampa `7 17 119`.

```
12 17 7 17 119
```

8 Tipi di dato

Lezione 8 – 7 ottobre 2026

8.1 Tipi e oggetti

I programmi devono memorizzare ed elaborare dati di tipo diverso, come interi e numeri in virgola mobile, in modi diversi: il compilatore deve quindi sapere che tipo di dato rappresenta un valore.

Definizione. In C un *oggetto* è una zona di memoria il cui contenuto può rappresentare valori. Gli oggetti che hanno un nome sono detti *variabili*. Il *tipo* di un oggetto determina:

- quanto spazio occupa in memoria;
- i valori che la variabile può assumere;
- le operazioni che si possono eseguire su di essa.

8.1.1 Tipi in C

- **Tipi di base**: interi (standard ed estesi) e floating-point (reali e complessi).
- **Tipi enumerati**.

- Il tipo `void`.
- **Tipi derivati**: puntatori, array, strutture, unioni, funzioni.

I tipi di base e i tipi enumerati formano i *tipi aritmetici*; i tipi aritmetici e i puntatori formano i *tipi scalari*; array e strutture sono detti *tipi aggregati*. Un *tipo funzione* descrive l'interfaccia di una funzione: il tipo del valore restituito ed eventualmente i tipi dei parametri.

8.2 Interi

I tipi interi con segno sono cinque (molti hanno sinonimi):

tipo	sinonimi
`signed char`	
`int`	`signed`, `signed int`
`short`	`short int`, `signed short`, `signed short int`
`long`	`long int`, `signed long`, `signed long int`
`long long` (C99)	`long long int`, `signed long long`, `signed long long int`

C definisce solo le dimensioni minime: `short` occupa almeno 2 byte, `long` almeno 4 e `long long` almeno 8. I tipi possono essere più grandi, ma devono rispettare l'ordine

$$\texttt{sizeof(short)} \leq \texttt{sizeof(int)} \leq \texttt{sizeof(long)} \leq \texttt{sizeof(long long)}.$$

8.2.1 Booleani

In C non esistono *true* e *false*: 0 è falso e qualsiasi valore diverso da 0 (ad esempio 1, −25, 123456) è vero.

```
1 if (3)                        if (0)
2     printf("YES\n");              printf("YES\n");
3 else                          else
4     printf("NO\n");               printf("NO\n");
```

Il primo `if` stampa `YES`, il secondo `NO`.

C99 ha introdotto il tipo intero senza segno `_Bool`: *vero* è codificato come 1, *falso* come 0. Includendo l'header `stdbool.h` si possono usare anche `bool`, `true` e `false`: `bool` è sinonimo di `_Bool` e `true`, `false` sono costanti uguali a 1 e 0.

8.2.2 Caratteri

Anche `char` è un tipo intero standard, ma il nome `char` è sinonimo di `signed char` oppure di `unsigned char` a seconda del compilatore. Occupa 1 byte e i caratteri corrispondono ai codici della tabella ASCII.

Con le variabili `char` si può fare aritmetica: sta al programmatore decidere se il numero contenuto è un codice di carattere o altro.

```
1 char ch = 'A';        // variabile di tipo char
2 printf("The character %c has the character code %d.\n", ch, ch);
3 printf("%c", ch + 1);
```

```
The character A has the character code 65.
B
```

8.3 Sistemi di numerazione

Definizione. Il sistema *binario* (base 2) rappresenta i numeri con due soli simboli, 0 e 1; ogni cifra è un *bit*. Quasi tutti i calcolatori lo usano internamente perché è semplice da realizzare con circuiti a porte logiche.

Definizione. Il sistema *esadecimale* (base 16) usa sedici simboli: le cifre 0–9 per i valori da zero a nove e le lettere `A, B, C, D, E, F` (o minuscole) per i valori da dieci a quindici. Ogni cifra esadecimale corrisponde a 4 bit (un *nibble*, metà di un byte da 8 bit): è una rappresentazione più leggibile dei valori binari. Il sistema *ottale* (base 8) usa le cifre 0–7.

Bin.	Esa.	Dec.	Bin.	Esa.	Dec.
0	0	0	1000	8	8
1	1	1	1001	9	9
10	2	2	1010	A	10
11	3	3	1011	B	11
100	4	4	1100	C	12
101	5	5	1101	D	13
110	6	6	1110	E	14
111	7	7	1111	F	15

8.3.1 Da binario/esadecimale a decimale

Un numero con cifre $d_n \dots d_1 d_0$ in base b vale $d_n b^n + d_{n-1} b^{n-1} + \cdots + d_1 b^1 + d_0 b^0$ (in base 2: $d_n 2^n + \cdots + d_1 2^1 + d_0 2^0 = N_{10}$).

Esempio. $1001_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 9$ e $\texttt{002B}_{16} = 2 \cdot 16 + 11 = 43$.

8.3.2 Da decimale a binario/esadecimale

Si divide ripetutamente il numero per la base e si leggono i resti *dal basso verso l'alto*. Per l'ottale si usa lo stesso algoritmo con base 8.

Esempio. 156 in binario (divisioni per 2): i quozienti successivi sono $78, 39, 19, 9, 4, 2, 1, 0$ e i resti $0, 0, 1, 1, 1, 0, 0, 1$. Letti dal basso: $156 = 10011100_2$.

1565 in esadecimale (divisioni per 16): $1565 = 97 \cdot 16 + 13$ (resto $13 = \texttt{d}$), $97 = 6 \cdot 16 + 1$ (resto 1), $6 = 0 \cdot 16 + 6$ (resto 6). Quindi $1565 = \texttt{61d}_{16}$.

Con N bit si rappresentano i numeri da 0 a $2^N - 1$; con 8 bit (un byte) da 0 a 255 (da `0000 0000` a `1111 1111`). Servono però anche i numeri negativi, e per questo esistono rappresentazioni diverse: segno e modulo, complemento a due.

8.4 Rappresentazione degli interi in memoria

8.4.1 Segno e modulo

Un bit (di solito il più a sinistra) indica il segno: 0 per i positivi, 1 per i negativi; i bit restanti rappresentano il modulo. Ad esempio `1001 1000` vale -24 (mentre se si usasse solo per numeri positivi varrebbe 152).

Con n bit si rappresentano i valori da $-(2^{n-1}-1)$ a $2^{n-1}-1$, e ± 0: con 8 bit da -127 a $+127$.

> **Osservazione.** Problema: lo zero ha due rappresentazioni, `0000 0000` ($+0$) e `1000 0000` (-0). Una soluzione è il complemento a due.

8.4.2 Complemento a due

Valore binario	Complemento a due	Senza segno
`00000000`	0	0
`00000001`	1	1
⋮	⋮	⋮
`01111110`	126	126
`01111111`	127	127
`10000000`	-128	128
`10000001`	-127	129
`10000010`	-126	130
⋮	⋮	⋮
`11111110`	-2	254
`11111111`	-1	255

Nel complemento a due esiste un solo zero, `00000000`. Per cambiare segno a un numero (positivo o negativo) si **invertono tutti i bit e poi si somma 1**.

Esempio.

- `0000 0101` (valore 5); invertendo: `1111 1010`; sommando 1: `1111 1011`, che vale -5.
- Da `1111 1011`: invertendo `0000 0100`, sommando 1 `0000 0101`, cioè di nuovo 5.

Con n bit si rappresentano i valori da -2^{n-1} a $2^{n-1}-1$: non c'è "-0", quindi si rappresenta un numero negativo in più. Con 8 bit, da -128 (`1000 0000`) a $+127$ (`0111 1111`).

> **Osservazione.** La regola "inverti e somma 1" non funziona per -128, perché 128 non è rappresentabile con 8 bit in complemento a due. D'ora in poi si usa il complemento a due.

8.4.3 Endianness

> **Definizione.** L'*endianness* indica l'ordine usato per interpretare numericamente una sequenza di byte della memoria come una parola più grande (e l'ordine di trasmissione dei byte su un collegamento digitale). Le parole possono essere in formato *big-endian* o *little-endian*, a seconda che si parta dall'estremità più significativa (big end) o meno significativa (little end).

Esempi: i mainframe IBM z/Architecture e i Motorola 68000 usano big-endian, i processori Intel x86 little-endian.

La memoria è un array di byte; ogni byte ha il suo *indirizzo logico* (un numero positivo), cioè l'indirizzo al quale un elemento appare risiedere dal punto di vista di un programma in esecuzione. Per architetture a 64 bit il limite superiore è $2^{64} - 1$.

Esempio. Il numero 2 scritto su 4 byte, a partire dall'indirizzo 1500:

	1500	1501	1502	1503
big-endian	00	00	00	02
little-endian	02	00	00	00

Esempio. Sulla stringa di 4 bit `0010`, in complemento a due: in big-endian vale 2; in little-endian si leggono i bit in ordine inverso (`0100`) e vale 4. Sulla stringa `1010`: in big-endian vale -6, in little-endian (`0101`) vale 5.

8.5 Dimensione e limiti dei tipi

L'operatore `sizeof` restituisce la dimensione esatta in byte di un tipo o di una variabile: `sizeof (tipo)` oppure `sizeof espressione` (in questo caso conta il tipo dell'espressione).

```c
int iIndex;
iIndex = 1000;
// sizeof(int) e sizeof(iIndex) valgono entrambi 4
```

I limiti dei tipi interi per il proprio compilatore sono nell'header `limits.h`, che definisce macro come `INT_MIN`, `INT_MAX`, `UINT_MAX`, `CHAR_MIN`, `CHAR_MAX`.

```c
#include <stdio.h>
#include <limits.h>

int main() {
    printf(" char %zu %d %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
    printf(" int %zu %d %d\n", sizeof(int), INT_MIN, INT_MAX);
    return 0;
}
```

```
char 1 -128 127
int 4 -2147483648 2147483647
```

Osservazione. Il risultato di `sizeof` ha tipo `size_t`: per stamparlo è corretto `%zu` (nelle slide compare `%d`). I valori stampati dipendono dalla piattaforma (qui un caso tipico).

8.6 Numeri in virgola mobile

Per rappresentare i non interi, con la virgola in qualsiasi posizione, C ha tre tipi floating-point standard:

- `float`: precisione singola;
- `double`: precisione doppia;
- `long double`: precisione estesa.

8.6.1 Precisione

Un valore in virgola mobile si memorizza solo con una *precisione* limitata, determinata dal formato binario e dalla memoria usata. La precisione si esprime in cifre significative: ad esempio con sei cifre significative la conversione in un numero decimale di sei cifre restituisce le sei cifre originali. La posizione della virgola e gli zeri iniziali o finali non contano: 123 456 000 e 0,00123456 si possono entrambi memorizzare in un tipo con precisione a sei cifre.

In C le operazioni aritmetiche tra numeri in virgola mobile sono eseguite internamente con precisione double o superiore.

```
1 float height = 1.2345, width = 2.3456;   // precisione singola
2 double area = height * width;            // il calcolo è fatto in double
```

Se il risultato è assegnato a una variabile `float`, viene arrotondato se necessario. L'header `float.h` definisce macro come `FLT_MIN`, `FLT_MAX` e `FLT_DIG`, che indicano intervallo di valori e precisione del tipo `float`.

8.6.2 Notazione E e formato IEEE 754

La *notazione E* è la rappresentazione testuale della notazione scientifica: `1.234e+56` significa $1{,}234 \times 10^{56}$.

Nel formato *IEEE 754* ogni numero finito è descritto da tre interi: un segno s (zero o uno), un *significando* (o mantissa) c e un esponente q. Il valore è

$$(-1)^s \times c \times b^q,$$

dove b è la base (2 o 10). Ad esempio con base 10, segno 1 (negativo), significando 12345 ed esponente -3 il valore è $-12{,}345$.

Il formato 754 a *precisione singola* usa: 1 bit di segno, 8 bit di esponente, 23 bit di significando. Il valore è

$$(-1)^{\text{segno}} \times \left(1 + \sum_{i=1}^{23} b_{23-i}\, 2^{-i}\right) \times 2^{(e-127)}.$$

Esempio. Con segno 0, esponente `01111100` ($e = 124$) e frazione `010...0` (cioè $b_{21} = 1$, tutti gli altri bit nulli) si ha $1 + 2^{-2} = 1{,}25$ e $2^{124-127} = 2^{-3}$, quindi

$$\text{valore} = (+1) \times 1{,}25 \times 2^{-3} = +0{,}15625.$$

Con la *precisione doppia* (`double`) si usano 1 bit di segno, 11 di esponente e 52 di frazione; la *precisione estesa* (`long double`, 80 bit) ha 1 bit di segno, 15 di esponente, 1 bit di parte intera e 63 di frazione.

8.6.3 Errori di arrotondamento

L'errore di arrotondamento è intrinseco nel calcolo in virgola mobile.

```
1 #include <stdio.h>
2
3 int main() {
4     int a = 16777217;
```

```
    float b = a;
    printf("%f\n", b);
}
```

```
16777216.000000
```

Il valore $16777217 = 2^{24} + 1$ non è rappresentabile esattamente con un `float`. Allo stesso modo conviene tenere in `double` anche il risultato di un prodotto come `height * width`: salvarlo in un `float` fa perdere precisione.

Osservazione. Il modo più semplice per evitare di accumulare errori è usare numeri in virgola mobile ad alta precisione, cioè `double` al posto di `float`. Sulle CPU moderne non c'è (quasi) penalizzazione di tempo, ma un `double` occupa il doppio della memoria.

8.7 Tipi enumerati

Le *enumerazioni* sono tipi interi definiti dal programmatore. La definizione inizia con la parola chiave `enum`, eventualmente seguita da un identificatore, e contiene l'elenco dei possibili valori con un nome per ciascuno:

```
enum [identificatore] { lista-enumeratori };
```

Esempio.

```
enum color { black, red, green, yellow, blue, white = 7, gray };
```

Le costanti elencate valgono $0, 1, 2, 3, 4, 7, 8$ (se non si assegna un valore, si continua dal precedente più uno).

```
enum color fgColor = blue,          // due variabili
           bgColor = yellow;        // di tipo enum color

void setFgColor(enum color fgc);    // funzione con parametro enum color
```

Con le variabili di tipo enumerato si può fare aritmetica ordinaria (`red + red` vale 2). Costanti diverse della stessa enumerazione possono avere lo stesso valore:

```
enum signals { OFF, ON, STOP = 0, GO = 1, CLOSED = 0, OPEN = 1 };
```

Esempio.

```
#include <stdio.h>

```

```c
enum week { sunday, monday, tuesday, wednesday, thursday, friday, saturday };

int main() {
    enum week today;
    today = wednesday;
    printf("Day %d", today + 1);
    return 0;
}
```

```
Day 4
```

Osservazione. Conviene usare sempre un'enumerazione quando una variabile (in particolare un parametro di funzione) può assumere solo uno tra pochi valori possibili: si evitano errori dovuti a costanti non valide, si documentano i valori ammessi ed è più mnemonico degli interi.

8.8 Il tipo void

Il tipo `void` indica che non è disponibile alcun valore. Non si possono dichiarare variabili o costanti di questo tipo, ma si usa:

- nelle dichiarazioni di funzioni;
- nelle espressioni `void`;
- nei puntatori a `void` (si vedranno con i puntatori).

Una funzione che non restituisce nulla ha tipo `void`: `void error(int a) {}`. `void` nella lista dei parametri di un prototipo indica che la funzione non ha parametri: `void printMenu(void) {}`. Il compilatore segnala un errore se si chiama `printMenu(3)`.

9 Costanti letterali

Lezione 9 – 12 ottobre 2026

9.1 Costanti letterali

Definizione. Un *literal* (costante letterale) è un token che denota un valore fisso, che può essere un intero, un numero in virgola mobile, un carattere o una stringa. Il tipo di un literal è determinato dal suo valore e dalla sua notazione.

9.2 Costanti intere

Una costante intera si può scrivere come un normale numero decimale oppure in notazione ottale o esadecimale. La notazione si specifica con un *prefisso*.

- **Decimale**: inizia con una cifra diversa da zero.
- **Ottale** (base 8, cifre $0, \ldots, 7$): inizia con uno zero iniziale. Il valore è $c_n 8^n + c_{n-1} 8^{n-1} + \cdots + c_1 8^1 + c_0 8^0$.
- **Esadecimale**: inizia con il prefisso `0x` o `0X`. Le cifre da A a F possono essere maiuscole o minuscole.

Esempio.

- $047 = 4 \cdot 8^1 + 7 \cdot 8^0 = 39$.
- `0xff`, `0Xff`, `0xFF` e `0XFF` valgono tutte $15 \cdot 16^1 + 15 \cdot 16^0 = 255$.

9.2.1 Tipo delle costanti intere

Il tipo di una costante è determinato nel momento in cui ne viene definito il valore. Le costanti intere degli esempi precedenti hanno di solito tipo `int`. Se il valore è troppo grande, il compilatore assegna il primo tipo, in una gerarchia, abbastanza grande da rappresentarlo. Ad esempio, la costante decimale `50000` ha tipo `int`, perché il massimo valore possibile per uno `short` (2 byte) è $32\,767 = 2^{15} - 1$.

Si può influenzare esplicitamente il tipo di una costante con un *suffisso*:

- `512U`: `unsigned int`;
- `0Xf0fUL`: `unsigned long`;
- `07771l`: `long long`;
- `123uLL`: `unsigned long long`.

Maiuscole e minuscole sono equivalenti (`ll` e `LL` sono la stessa cosa).

```
1 int a = 512U;
2 int b = 1LL;
3 int c = 7UL;
```

9.3 Costanti in virgola mobile

Le costanti in virgola mobile si scrivono in notazione decimale o esadecimale. Una costante è una sequenza di cifre decimali contenente un punto decimale. Si può moltiplicare il valore per una potenza di 10, come nella notazione scientifica: la potenza di 10 si indica con un esponente introdotto dalla lettera `e` o `E`.

Esempio. `2.34E5` vale $2.34 \cdot 10^5$ e `67e-12` vale $67.0 \cdot 10^{-12}$.

Il punto decimale può anche essere il primo o l'ultimo carattere: `10.` e `.234E6` sono numeri validi. Il numero `10` senza punto decimale è invece una costante *intera*, non in virgola mobile.

9.3.1 Forzare il tipo

Il tipo di default di una costante in virgola mobile è `double`. Con il suffisso `F` o `f` si assegna il tipo `float`; con il suffisso `L` o `l` il tipo `long double`.

9.4 Costanti carattere

Una costante carattere è un carattere racchiuso tra apici singoli: `'a'`, `'0'`, `'*'`. Per rappresentare l'apice, il backslash e il ritorno a capo si usano `'\''`, `'\\'` e `'\n'`. Le costanti carattere hanno tipo `int`.

9.4.1 Sequenze di escape

Sequenza	Valore del carattere	Azione sul dispositivo di output
`\'`	apice singolo (')	stampa il carattere
`\"`	virgolette (")	stampa il carattere
`\?`	punto interrogativo	stampa il carattere
`\\`	backslash	stampa il carattere
`\a`	alert	produce un segnale acustico o visivo
`\b`	backspace	riporta la posizione attiva indietro di un carattere
`\f`	form feed	porta la posizione all'inizio della pagina successiva
`\n`	line feed	porta la posizione all'inizio della riga successiva
`\r`	carriage return	porta la posizione all'inizio della riga corrente

9.5 Stringhe letterali

Una *stringa letterale* è una sequenza di caratteri (e/o sequenze di escape) racchiusa tra virgolette doppie, ad esempio `"Hello world!\n"`. Come le costanti carattere, può contenere tutti i caratteri del set di caratteri sorgente.

Una stringa letterale è un *array statico di char* che contiene i codici dei caratteri seguiti dal terminatore di stringa, il carattere nullo `\0`. La stringa vuota `""` occupa esattamente un byte di memoria, che contiene il carattere nullo di terminazione.

```
1 char helloWorld[128] = "Hello World!\n";
2 printf("Print the string: %s\n", helloWorld);
```

Riferimenti

Sul libro: sezione 15.6 (pagina 339) e pagina 46, sezione 9.10.

10 Conversioni di tipo

Lezione 10 – 13 ottobre 2026

10.1 Conversioni

Operandi di tipo diverso si possono combinare in una stessa operazione.

```
1 double dVar = 2.5;
2 dVar = dVar * 3;
3 if (dVar < 10L) { /* ... */ }
```

Quando gli operandi hanno tipi diversi, il compilatore cerca di convertirli a un tipo uniforme prima di eseguire l'operazione.

10.1.1 Conversioni implicite

Il compilatore applica conversioni di tipo implicite quando

- gli operandi hanno tipi non corrispondenti;
- si chiama una funzione con un argomento il cui tipo non corrisponde a quello del parametro;

- si inizializzano variabili o si assegnano loro valori.

Se la conversione necessaria non è possibile il compilatore segnala un errore; altre volte segnala un warning.

10.1.2 Operatore di cast

Si può convertire un valore da un tipo a un altro anche *esplicitamente*, con l'operatore di cast:

```
(nome_tipo) espressione
```

```
1 int sum = 10, count = 3;
2 double mean = (double) sum / count;   // 3.3333
```

Il valore di `sum` viene prima convertito (esplicitamente) in `double`; il compilatore deve poi convertire implicitamente il divisore, il valore di `count`, allo stesso tipo prima di eseguire la divisione.

Osservazione. Conviene usare sempre il cast quando c'è la possibilità di perdere informazione. I cast espliciti evitano i warning del compilatore.

10.2 Conversione dei tipi aritmetici

10.2.1 Gerarchia dei tipi

Quando gli operandi aritmetici hanno tipi diversi, la conversione implicita è governata dal *rango di conversione* dei tipi:

- due tipi interi senza segno qualsiasi hanno ranghi diversi: se uno è più largo dell'altro ha rango più alto;
- ogni tipo intero con segno ha lo stesso rango del corrispondente tipo senza segno;
- i tipi interi standard sono ordinati: `_Bool` < `char` < `short` < `int` < `long` < `long long`;
- i tipi in virgola mobile sono ordinati: `float` < `double` < `long double`;
- il tipo in virgola mobile di rango più basso, `float`, ha rango più alto di qualunque tipo intero;
- gli `enum` hanno lo stesso rango di `int`.

10.2.2 Promozione intera

In qualunque espressione si può usare un valore di tipo con rango inferiore a `int` al posto di un operando di tipo `int` o `unsigned int`. In questi casi il compilatore applica la *promozione intera*: ogni operando di rango inferiore a `int` viene convertito automaticamente in `int`, purché `int` sia in grado di rappresentare tutti i valori del tipo originale; altrimenti viene convertito in `unsigned int`. Le operazioni nella CPU vengono eseguite su almeno 4 byte.

Esempio.

```
1 #include <stdio.h>
2 int main()
3 {
4     char a = 30, b = 40, c = 10;
```

```
    char d = (a * b) / c;
    printf("%d ", d);
    return 0;
}
```

```
120
```

A prima vista `(a*b)/c` sembra causare un overflow aritmetico: i `char` con segno vanno da -128 a 127 (nella maggior parte dei compilatori) e il valore di `(a*b)` è 1200. Ma sui tipi `char` avviene la promozione intera, quindi si ottiene il risultato corretto senza overflow.

10.2.3 Conversioni aritmetiche usuali

Le *conversioni aritmetiche usuali* sono le conversioni implicite applicate automaticamente agli operandi di tipo aritmetico diverso per la maggior parte degli operatori. Si applicano implicitamente a

- gli operatori aritmetici a due operandi: `*`, `/`, `%`, `+` e `-`;
- gli operatori relazionali e di uguaglianza: `<`, `<=`, `>`, `>=`, `==` e `!=`.

Esempio.

```
int x = 0;
int i = -1;
unsigned int limit = 200U;
long n = 30L;

if (i < limit)
    x = limit * n;

printf("%d\n", x);
```

```
0
```

Le conversioni aritmetiche usuali si applicano così:

- Se uno degli operandi ha tipo in virgola mobile, l'operando di rango più basso viene convertito a un tipo con lo stesso rango dell'altro. I tipi reali si convertono solo in tipi reali.
- Se entrambi gli operandi sono interi, si esegue prima la promozione intera su entrambi. Se dopo la promozione i tipi sono ancora diversi, si procede così:
  - **Regola 1**: se un operando ha un tipo *senza segno* T con rango almeno pari a quello del tipo dell'altro operando, l'altro operando viene convertito al tipo senza segno T.
  - **Regola 2**: altrimenti, un operando ha un tipo *con segno* T con rango più alto di quello dell'altro. L'altro operando viene convertito al tipo con segno T solo se T è in grado di rappresentare tutti i valori del suo tipo precedente; se non lo è, entrambi gli operandi vengono convertiti al tipo senza segno corrispondente a T.

Esempio (Applicazione delle regole)**.** Nell'esempio sopra, per valutare il confronto `i < limit` il valore di `i`, -1, viene convertito a `unsigned int` (Regola 1: `limit` è senza segno e ha rango almeno pari a `int`). Il risultato è un numero positivo molto grande, quindi la condizione dell'`if` è *falsa* e `x` resta 0.

In `limit * n` (Regola 2) `limit` viene convertito al tipo di `n`, `long`, se l'intervallo dei valori di `long` contiene tutto l'intervallo di `unsigned int`. Altrimenti (ad esempio se `int` e `long` hanno entrambi 32 bit) entrambi i fattori vengono convertiti a `unsigned long`.

Come si rappresenta -1 in un `int` (little endian, complemento a due)? Con 32 bit tutti a 1: `11111111111111111111111111111111`. Un `unsigned int` ha 32 bit (in quel compilatore), quindi -1 viene convertito implicitamente in `unsigned int` e il suo valore diventa 4 294 967 295.

10.3 Altre conversioni implicite

Il compilatore converte automaticamente i valori aritmetici anche nei casi seguenti:

- nelle assegnazioni e nelle inizializzazioni, il valore dell'operando di destra viene sempre convertito al tipo dell'operando di sinistra;
- nelle chiamate di funzione, gli argomenti vengono convertiti ai tipi dei parametri corrispondenti;
- nelle istruzioni `return`, il valore dell'espressione viene convertito al tipo restituito dalla funzione.

In un'*assegnazione composta*, come `x += 2.5` (cioè `x = x + 2.5`), i valori di entrambi gli operandi sono prima soggetti alle conversioni aritmetiche usuali; poi il risultato dell'operazione viene convertito, come in una assegnazione semplice, al tipo dell'operando di sinistra. Se `x` è `int`, viene convertito in `double` e poi il risultato `x + 2.5` (di tipo `double`) viene riconvertito in `int`.

Esempio.

```
#include <math.h>   // dichiara double sqrt(double)
int i = 7;
float x = 0.5;      // la costante viene convertita da double a float

i = x;              // il valore di x viene convertito da float a int

x += 2.5;           // prima dell'addizione x viene convertito in double;
                    // la somma viene poi convertita in float per l'assegnazione

x = sqrt(i);        // l'argomento viene convertito da int a double;
                    // il valore restituito da double a float

long my_func() {    // la costante 0 viene convertita a long,
    /* ... */       // il tipo restituito dalla funzione
    return 0;
}
```

10.4 Conversioni verso tipi interi

10.4.1 Verso tipi interi senza segno

I valori interi sono sempre preservati se rientrano nell'intervallo del nuovo tipo senza segno, cioè tra 0 e `Utype_MAX`. Per i valori fuori intervallo, il valore dopo la conversione si ottiene sommando

o sottraendo `Utype_MAX` + 1 tante volte quante servono perché il risultato rientri nell'intervallo del nuovo tipo.

```
unsigned short n = 1000;    // 1000 rientra nell'intervallo di unsigned short: ok
n = -1;                     // il valore -1 deve essere convertito
```

Si ha $-1 + (\texttt{USHRT_MAX} + 1) = \texttt{USHRT_MAX}$, quindi l'ultima istruzione equivale a `n = USHRT_MAX;`.

10.4.2 Da virgola mobile a interi

Per convertire un numero reale in virgola mobile in un tipo intero (con o senza segno) il compilatore scarta la parte frazionaria. Se la parte intera rimanente è fuori dall'intervallo del nuovo tipo, il risultato della conversione è *indefinito*.

```
double x = 2.9;
unsigned long n = x;    // la parte frazionaria di x viene persa: n = 2
```

10.4.3 Verso tipi interi con segno

Il problema del superamento dell'intervallo dei valori del tipo di destinazione può presentarsi anche quando un valore intero, con o senza segno, viene convertito in un tipo intero *con segno*, ad esempio da `long` o `unsigned int` a `int`. A differenza delle conversioni verso tipi senza segno, il risultato di un overflow in conversione verso un tipo con segno è lasciato all'implementazione. La maggior parte dei compilatori scarta i bit più significativi della rappresentazione binaria del valore originale e interpreta i bit più bassi secondo il nuovo tipo.

Esempio.

```
#include <stdio.h>
#include <limits.h>
int main() {
    long long int a = (LLONG_MAX - UINT_MAX) + 1;
    int b = a;

    printf("b: %d\n", b);
    printf("a: %lld\n", a);
}
```

```
b: 1
a: 9223372032559808513
```

In esadecimale `LLONG_MAX` è `0x7FFFFFFFFFFFFFFF` e `UINT_MAX` è `0xFFFFFFFF`, quindi

$$a = \texttt{0x7FFFFFFF00000001}.$$

Convertendo in `int` (32 bit) si tengono solo i 32 bit più bassi, `00000001`: perciò $b = 1$.

10.5 Conversioni verso tipi in virgola mobile

Non tutti i valori interi sono rappresentabili esattamente nei tipi in virgola mobile. Ad esempio, benché l'intervallo di valori del tipo `float` includa quello di `long` e `long long`, `float` ha una precisione di sole sei cifre decimali. Quindi alcuni valori `long` non possono essere memorizzati esattamente in un `float`: il risultato della conversione è il valore rappresentabile immediatamente inferiore o superiore.

Esempio.

```
1 float r_var = 16777217;
2 double l_var = 16777217;
3 printf("The rounding error (l_var - r_var) is %.2f\n", l_var - r_var);
4 printf("r_var is %.2f\n", r_var);
```

```
The rounding error (l_var - r_var) is 1.00
r_var is 16777216.00
```

Ogni valore di un tipo in virgola mobile si può rappresentare esattamente in un altro tipo in virgola mobile di precisione maggiore. Quindi, quando un `double` viene convertito in `long double`, o un `float` in `double` o `long double`, il valore è esattamente preservato.

Nelle conversioni da un tipo più preciso a uno meno preciso, invece, il valore da convertire può essere fuori dall'intervallo del nuovo tipo.

- Se il valore supera l'intervallo del tipo di destinazione, il risultato della conversione è *indefinito*.
- Se il valore è nell'intervallo ma non è rappresentabile esattamente con la precisione del tipo di destinazione, il risultato è il valore rappresentabile immediatamente inferiore o superiore.

Riferimenti

Niente sul libro. Risorse indicate dal docente:

- `https://www.w3schools.com/c/c_type_conversion.php`
- `https://www.guru99.com/c-type-casting.html`
- soprattutto, il capitolo 4 di *C in a Nutshell* (O'Reilly): `https://www.oreilly.com/library/view/c-in-a/0596006977/ch04.html`